

Entrega #1 Mini-lenguaje

Jesús Ismael Ramos Recio

Herramientas de generación automática

Las herramientas de generación automática son fundamentales en la construcción de compiladores e intérpretes que automatizan procesos complejos y reducen errores para ahorrar tiempo en desarrollo. En esta sección se listan 3 de estas herramientas:

- **Flex y Bison:**
Estas funcionan principalmente con C y C++. Flex se encarga de generar analizadores léxicos mediante expresiones regulares basadas en autómatas. Bison se encarga de generar analizadores sintácticos mediante gramáticas libres de contexto.
- **ANTLR:**
Esta es una herramienta moderna y multiplataforma basada en Java. Lo que hace es generar analizadores sintácticos a partir de gramáticas y construir árboles de análisis. Esta ofrece mayor flexibilidad mediante listeners y visitors.
- **PLY (Python Lex-Yacc):**
Este está implementado completamente en Python. Esta herramienta combina el análisis léxico y sintáctico. Gracias a la reflexión de Python, es fácil de usar e integrar y permite definir reglas directamente en el código.

Para este proyecto, se escogió la herramienta de PLY. Esto porque PLY, además de lo simple que es su uso en Python, es más práctico y directo para proyectos pequeños. También ayuda a hacer el desarrollo de un compilador rápido y hay gran claridad del código.

Scanner y Parser

En esta sección se muestra el desarrollo del analizador. Se presentará primero el analizador léxico, o Scanner, y luego el analizador sintáctico, o Parser.

Scanner

```
# Uso de la herramienta
import ply.lex as lex # type: ignore

# Lista de los Tokens
tokens = [
    'PROGRAM', 'FUNC', 'BODY', 'START', 'END',
    'IF', 'ELSE', 'WHILE', 'CALL', 'PRINT', 'INT',
    'FLOAT', 'NULL', 'VOID', 'VAR', 'ID', 'CTE_INT',
    'CTE_FLOAT', 'ASIGNA', 'MAS', 'MENOS', 'MULT', 'DIV',
    'IGUAL', 'NO_IGUAL', 'MENOR', 'MAYOR', 'MENOR_IGUAL',
    'MAYOR_IGUAL', 'PARENT_A', 'PARENT_C', 'CORCH_A', 'CORCH_C',
    'LLAVE_A', 'LLAVE_C', 'PUNTO_COMA', 'COMA', 'DOS_PUNTOS'
]

# Formato de tokens: t_NOMBRE = r'expresion_regular'
```

```
# Diccionario de palabras reservadas
```

```
reserved = {  
    'program': 'PROGRAM',  
    'func': 'FUNC',  
    'body': 'BODY',  
    'start': 'START',  
    'end': 'END',  
    'if': 'IF',  
    'else': 'ELSE',  
    'while': 'WHILE',  
    'call': 'CALL',  
    'print': 'PRINT',  
    'int': 'INT',  
    'float': 'FLOAT',  
    'null': 'NULL',  
    'void': 'VOID',  
    'var': 'VAR'  
}
```

```
# Simbolos y operadores
```

```
t_ASIGNA = r'='  
t_MAS = r'\+'  
t_MENOS = r'-'  
t_MULT = r'\*'  
t_DIV = r'/'  
t_IGUAL = r'=='  
t_NO_IGUAL = r'!='  
t_MENOR = r'<'  
t_MAYOR = r'>'  
t_MENOR_IGUAL = r'<='  
t_MAYOR_IGUAL = r'>='  
t_PARENT_A = r'\('  
t_PARENT_C = r'\)'  
t_CORCH_A = r'\['  
t_CORCH_C = r'\]'  
t_LLAVE_A = r'\{'  
t_LLAVE_C = r'\}'  
t_PUNTO_COMA = r';'  
t_COMA = r','  
t_DOS_PUNTOS = r':'
```

```
# Definir flotantes
def t_CTE_FLOAT(t):
    r'[0-9]+\.[0-9]+'
    t.value = float(t.value)
    return t

# Definir enteros
def t_CTE_INT(t):
    r'[0-9]+'
    t.value = int(t.value)
    return t

# Definir identificadores y si son palabras reservadas
def t_ID(t):
    r'[a-zA-Z_][a-zA-Z0-9_]*'
    t.type = reserved.get(t.value, 'ID') # Checar si es una palabra
reservada
    return t

# Ignorar espacios y tabulaciones
t_ignore = ' \t'

# Manejar nuevas líneas
def t_newline(t):
    r'\n+'
    t.lexer.lineno += len(t.value)

# Manejar errores de lexico
def t_error(t):
    print(f"Error lexico: Caracter '{t.value[0]}' invalido en linea
{t.lexer.lineno}")
    t.lexer.skip(1)

lexico = lex.lex()
```

Parser

```
# Formato para definir reglas
def p_var(p):
    '''
    var : VAR tipo lista_id PUNTO_COMA
    '''
    print(f" Variable(s): {p[3]} tipo {p[2]}")
    p[0] = None
```

Desglose de las referencias para la regla “var”

p[1]: VAR
p[2]: tipo
p[3]: lista_id
p[4]: PUNTO_COMA

Este formato se sigue en todas las reglas gramaticales

Parser completo:

```
import ply.yacc as yacc
from lexico import tokens, lexico

precedence = (
    ('left', 'IGUAL', 'NO_IGUAL', 'MENOR', 'MAYOR', 'MENOR_IGUAL',
    'MAYOR_IGUAL'),
    ('left', 'MAS', 'MENOS'),
    ('left', 'MULT', 'DIV'),
    ('right', 'UMENOS'),
)

def p_program(p):
    '''
    programa : PROGRAM ID DOS_PUNTOS vars func START body END
    '''
    print("Programa valido")
    print(f" Nombre: {p[2]}")
    p[0] = ('program', p[2])

def p_func(p):
    '''
    func : VOID ID PARENT_A parametros PARENT_C vars START body END
        | INT ID PARENT_A parametros PARENT_C vars START body END
        | FLOAT ID PARENT_A parametros PARENT_C vars START body END
        | empty
```

```

'''
    if len(p) > 2:
        print(f" Funcion: {p[2]} tipo {p[1]}")
    p[0] = None

def p_parametros(p):
    '''
        parametros : tipo mas_parametros
                    | empty
    '''
    if len(p) == 3:
        p[0] = [p[1]] + (p[2] if p[2] else [])
    else:
        p[0] = []

def p_mas_parametros(p):
    '''
        mas_parametros : COMA tipo mas_parametros
                    | empty
    '''
    if len(p) == 4:
        p[0] = [p[2]] + (p[3] if p[3] else [])
    else:
        p[0] = []

def p_tipo(p):
    '''
        tipo : INT
              | FLOAT
    '''
    p[0] = p[1]

def p_vars(p):
    '''
        vars : var vars
              | empty
    '''
    p[0] = None

def p_var(p):
    '''
        var : VAR tipo lista_id PUNTO_COMA
    '''

```

```

print(f" Variable(s): {p[3]} tipo {p[2]}")
p[0] = None

def p_lista_id(p):
    '''
    lista_id : ID
              | ID COMA lista_id
    '''
    if len(p) == 2:
        p[0] = [p[1]]
    else:
        p[0] = [p[1]] + p[3]

def p_body(p):
    '''
    body : estatuto body
          | empty
    '''
    p[0] = None

def p_estatuto(p):
    '''
    estatuto : asigna
              | condicion
              | ciclo
              | call
              | print
    '''
    if p[1] is not None:
        p[0] = p[1]
    else:
        p[0] = None

def p_asigna(p):
    '''
    asigna : ID ASIGNA exp PUNTO_COMA
    '''
    print(f" Asigna: {p[1]} = {p[3]}")
    p[0] = ('asigna', p[1], p[3])

def p_condicion(p):
    '''
    condicion : IF PARENT_A exp_relacion PARENT_C estatuto
    '''

```

```

        | IF PARENT_A exp_relacion PARENT_C estatuto ELSE
estatuto
'''
    if len(p) == 6:
        print(f" If: condicion {p[3]}")
        p[0] = ('if', p[3], p[5])
    else:
        print(f" If-Else: condicion {p[3]}")
        p[0] = ('if-else', p[3], p[5], p[7])

def p_ciclo(p):
    '''
    ciclo : WHILE PARENT_A exp_relacion PARENT_C body
    '''
    print(f" While: condicion {p[3]}")
    p[0] = ('while', p[3], p[5])

def p_call(p):
    '''
    call : ID PARENT_A lista_exp PARENT_C PUNTO_COMA
    '''
    print(f" Call: {p[1]}({p[3]})")
    p[0] = ('call', p[1], p[3])

def p_print(p):
    '''
    print : PRINT PARENT_A exp PARENT_C PUNTO_COMA
    '''
    print(f" Print: {p[3]}")
    p[0] = ('print', p[3])

def p_lista_exp(p):
    '''
    lista_exp : exp mas_exp
                | empty
    '''
    if len(p) == 3:
        p[0] = [p[1]] + (p[2] if p[2] else [])
    else:
        p[0] = []

def p_mas_exp(p):
    '''

```

```

mas_exp : COMA exp mas_exp
        | empty
        '''

if len(p) == 4:
    p[0] = [p[2]] + (p[3] if p[3] else [])
else:
    p[0] = []

def p_exp(p):
    '''
    exp : termino sig_exp
    '''
    p[0] = p[1]

def p_sig_exp(p):
    '''
    sig_exp : MAS termino sig_exp
            | MENOS termino sig_exp
            | empty
    '''
    if len(p) == 4:
        p[0] = (p[1], p[2], p[3])
    else:
        p[0] = None

def p_termino(p):
    '''
    termino : factor sig_termino
    '''
    p[0] = p[1]

def p_sig_termino(p):
    '''
    sig_termino : MULT factor sig_termino
                | DIV factor sig_termino
                | empty
    '''
    if len(p) == 4:
        p[0] = (p[1], p[2], p[3])
    else:
        p[0] = None

```



```

def p_factor(p):
    '''
    factor : ID
            | CTE_INT
            | CTE_FLOAT
            | NULL
            | PARENT_A exp PARENT_C
            | MENOS factor %prec UMENOS
    '''
    if len(p) == 2:
        p[0] = p[1]
    elif len(p) == 4 and p[1] == '(':
        p[0] = p[2]
    elif len(p) == 3:
        p[0] = ('-', p[2])

def p_exp_relacion(p):
    '''
    exp_relacion : exp op_relacion exp
    '''
    p[0] = (p[2], p[1], p[3])

def p_op_relacion(p):
    '''
    op_relacion : IGUAL
                | NO_IGUAL
                | MENOR
                | MAYOR
                | MENOR_IGUAL
                | MAYOR_IGUAL
    '''
    p[0] = p[1]

def p_empty(p):
    '''
    empty :
    '''
    p[0] = None

def p_error(p):
    if p:
        print(f"Error sintactico: token '{p.value}' inesperado en linea {p.lineno}")

```

```
        else:
            print("Error sintactico: fin de archivo inesperado")

parser = yacc.yacc()
```

Main

```
import sys
from lexico import lexico
from parser import parser

def main():

    filename = sys.argv[1]
    try:
        with open(filename, 'r', encoding='utf-8') as f:
            code = f.read()

        print(f"Analizando archivo: {filename}")
        print("-" * 40)
        result = parser.parse(code, lexer=lexico)
        print("-" * 40)

        if result is not None:
            print("Análisis exitoso.")
        else:
            print("El análisis presenta advertencias.")

    except FileNotFoundError:
        print(f"Error: Archivo '{filename}' no encontrado.")
        sys.exit(1)
    except Exception as e:
        print(f"Error: {e}")
        sys.exit(1)

if __name__ == "__main__":
    main()
```

Test Cases

En esta sección se muestran 3 test cases que se llevaron a cabo para validar el funcionamiento del analizador.

Test Case	Descripción	Código de prueba	Salida
Print de un valor	El analizador reconoce correctamente la instrucción de imprimir un valor entero	program print_valor: start print(42); end	Salida: Print: 42
Print de múltiples variables	El analizador reconoce las declaraciones de diferentes variables con diferentes valores y la impresión de cada una de ellas.	program multi_var: var int a, b, c; start a = 1; b = 2.45; c = hola; print(a); print(b); print(c); end	Variable(s): ['a', 'b', 'c'] tipo int Asigna: a = 1 Asigna: b = 2.45 Asigna: c = hola Print: a Print: b Print: c
Conteo de una variable	El analizador reconoce la declaración de una variable entera y la estructura de un ciclo. Dentro de dicho ciclo, también reconoce la instrucción de hacer un ciclo de sumar por 1 la variable hasta ser igual que la constante definida para detener el ciclo.	program contar: var int x; start x = 0; while (x < 5) x = x + 1; print(x); end	Variable(s): ['x'] tipo int Asigna: x = 0 Asigna: x = x Print: x While: condicion ('<', 'x', 5)